

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Szabó-Sáfár Benedek
2026

Szegedi Tudományegyetem
Informatikai Intézet

**EEG hullámok vizsgálata gépi tanulási
módszerekkel**

Szakdolgozat

Készítette
Szabó-Sáfár Benedek
programtervező
informatikus szakos
hallgató

Témavezető
Dr. Németh Gábor
adjunktus

Szeged
2026

Feladatkiírás

A témát kiíró oktató neve: Németh Gábor, társ témavezető: Dr. Devosa Iván

A témát meghirdető tanszék: Képfeldolgozás és Számítógépes Grafika Tanszék

Típus: Szakdolgozat

Ki jelentkezhethet: 1 fő, Programtervező informatikus BSc, Mérnök-informatikus BSc, vagy Gazdaságinformatikus BSc szakos hallgató

A feladat rövid leírása:

Egy kutatás során vizsgálták, hogy egészséges és idegrendszeri problémákkal, kényszermozgással küzdő „beteg” emberek hogyan reagálnak a zene ritmusára.

A jelentkező feladata gépi tanuló és statisztikai módszerekkel vizsgálni az egészséges és „beteg” személyek EEG adatait és megállapítani, hogy az EEG adatok vizsgált tulajdonságaiból lehet-e következtetni az idegrendszeri problémák jelenlétére.

Szakirodalom: magyar és angol nyelvű szakirodalom áll rendelkezésre

Előismeretek: nem szükségesek

Tartalmi összefoglaló

- **A téma megnevezése:**

EEG-alapú beteg–kontroll klasszifikáció gépi tanulási módszerekkel

- **A megadott feladat megfogalmazása:**

Olyan adatfeldolgozó és osztályozórendszer fejlesztése, amely zenehallgatás közben rögzített EEG-felvételek alapján képes megkülönböztetni halmozottan fogyatékos és egészséges személyeket. A rendszernek grafikus felhasználói felülettel kell rendelkeznie, amely lehetővé teszi a modellek betanítását és új felvételek osztályozását.

- **A megoldási mód:**

A nyers EEG-adatokból jellemzőkinyerési módszerekkel számvektorokat állítottam elő, amelyek alapján több gépi tanulási modellt tanítottam be és hasonlítottam össze. A rendszer grafikus felhasználói felületen keresztül kezelhető, és kiegészítő elemző szkriptekkel egészül ki.

- **Alkalmazott eszközök, módszerek:**

Python programozási nyelv; scikit-learn, XGBoost, numpy, pandas, matplotlib könyvtárak; statisztikai jellemzőkinyerés és fázis spektrum elemzés; Support Vector Machine, Random Forest, Random Forest MLP és XGBoost algoritmusok; 5-szörös rétegzett keresztvalidáció; GridSearch hiperparaméter-optimalizálás; kétmintás t-teszt; PCA dimenziócsökkentés; tkinter grafikus felület.

- **Elért eredmények:**

A legjobb eredményt az RF+MLP és az SVM modellek érték el Statistical jellemzőkinyerési módszerrel, mindkettő 90%-os pontossággal. A kétmintás t-teszt alapján a beteg és egészséges csoport közötti különbség több csatornán és epochban statisztikailag szignifikáns, különösen az alpha1 és beta2 sávokban.

- **Kulcsszavak:**

EEG, gépi tanulás, osztályozás, jellemzőkinyerés, Support Vector Machine, Random Forest, XGBoost, fázis spektrum, keresztvalidáció, klasszifikáció

Tartalomjegyzék

<i>Feladatkiírás</i>	3
Tartalmi összefoglaló	4
Bevezetés	6
1. Adatok és módszertan.....	8
1.1 Adatok bemutatása	8
1.2 Az EEG és frekvenciasávok	9
1.3 Jellemzőkinyerési módszerek.....	10
1.4 Gépi tanulási algoritmusok	12
1.5 Hiperparaméter-optimalizálás és keresztvalidáció	13
1.6 Kiértékelési metrikák.....	14
2. Implementáció	16
2.1 Projekt felépítése	16
2.2 Jellemzőkinyerés – feature_extraction.py	17
2.3 Modellek betanítása	19
2.4 Epoch analízis – epoch_analysis.py.....	21
2.5 Jellemzőkiválasztás – feature_selection.py	22
2.6 Vizualizáció – visualization.py	23
2.7 Grafikus felhatalnálói felület – gui.py	24
3. Eredmények	25
3.1 A modellek teljesítményének összehasonlítása.....	25
3.2 Jellemzők fontossága	27
3.3 Feature selection eredményei	28
3.4 Epoch analízis eredményei.....	30
3.5 Döntési határok és jellemzők vizualizációja	31
3.6 PCA vizualizáció	33
4. Felhatalnálói és telepítési útmutató	35
4.1 Rendszerkövetelmények	35
4.2 Telepítési útmutató.....	35
4.3 Grafikus felület használata	36
4.4 Az elemző szkriptek használata.....	37
5. Összefoglalás.....	38
6. Nyilatkozat AI használatáról.....	40
Irodalomjegyzék.....	41
Nyilatkozat	42

Bevezetés

Az agy elektromos aktivitásának mérése az elmúlt évtizedekben komoly teret nyert az orvostudományban és az idegtudományi kutatásokban. Az EEG lényege egyszerű: non-invazív módon, milliszekundumos időfelbontással rögzíti az agy elektromos jeleit. Ez teszi alkalmassá arra, hogy kognitív és motoros folyamatokat is nyomon lehessen vele követni. Engem informatikusként az ilyen feladatok vonzanak a legjobban, ahol a gépi tanulás nem öncélúan jelenik meg, hanem egy valódi kérdés megválaszolására szolgál. Olyasvalamire, aminek tétje van.

A projekt elején még nem a végleges adatokkal dolgoztam, Parkinson-kóros betegek EEG-felvételeit vizsgáltam [1], és az volt a cél, hogy epochonként, vagyis időszeletenként meghatározzam, a személy éppen ír, gondolkodik, vagy semmit nem csinál. A fájlok .set formátumban voltak, és többféle jellemzőkinyerési megközelítést is kipróbáltam. Egyértelmű eredmény nem született belőle, viszont rengeteget tanultam: megismertem az EEG-adatok szerkezetét, és összeállt bennem, hogyan kell egy gépi tanulási pipeline-t felépíteni nulláról, ami a dolgozatomban később nagyon jól jött.

A tényleges projekt adatokat témavezetőmtől kaptam meg, ezek Dr. Tiszai Luca kutatásához kapcsolódnak [2], aki halmozottan fogyatékos személyek zenei befogadását vizsgálja. A kutatásai során egy figyelemreméltó dolgot figyelt meg: a vizsgált személyek zenehallgatás közben apró, de azonosítható mozdulatokkal reagálnak bizonyos zenei eseményekre. Egy Bach-mű váratlan harmóniaváltására például, vagy Bartók ritmusainak sajátosságaira. Ez azt sejteti, hogy a zenefeldolgozás náluk is aktivál valamiféle memória- és anticipációs mechanizmust. Ha ez valóban így van, akkor az komolyan megkérdőjelezi az IQ-alapú kognitív besorolások érvényességét. A kutatás egyik fő célja ennek bizonyítása, ehhez viszont mérhetővé kell tenni a jelenséget, ami nagy mennyiségű EEG-adat automatizált feldolgozását igényli.

Az adatállomány összesen 60 felvételből áll. 26 betegtől és 25 egészséges személytől származnak a zenehallgatás közben rögzített EEG-jelek, plusz 9 felvétel el lett különítve manuális tesztelésre. Ezeket a modellt a tanítás során soha nem látta, szóval a grafikus felületen valódi független tesztetként lehet őket használni. Fejlesztés előtt utánanéztam, milyen megközelítések léteznek hasonló osztályozási feladatokra. EEG-alapú beteg-kontroll osztályozást epilepsziánál[3], depresszióánál[4] és Alzheimer-kóránál[5] már vizsgáltak, viszont zenehallgatáshoz kötött adatokon elvégzett ilyen munka

szinte alig található az irodalomban. Ez egyrészt nehezítette a tájékozódást, másrészt pont ezért volt izgalmas az egész: nem volt kész recept, saját megközelítést kellett kitalálni.

A rendszer három modellt alkalmaz: Support Vector Machine-t, Random Forest-et és XGBoost-ot, valamint ezek egy kombinált változatát, ahol a Random Forest valószínűségi kimenetei egy neurális háló bemenetül szolgálnak. Mindegyik a kinyert jellemzők alapján próbálja szétválasztani a beteg és egészséges személyek felvételeit, és az eredményeiket egymással is összehasonlítom. Készült hozzá egy grafikus felhasználói felület is, ahol a betanítás, a vizualizáció és az új felvételek elemzése elvégezhető programozási tudás nélkül is. A dolgozat szerkezete ennek megfelelően alakul: a feladatkiírás után az adatok és módszerek bemutatása következik, ahol a jellemzőkinyerési eljárásokat és a gépi tanulási megközelítéseket ismertetem. Az implementáció fejezete a rendszer felépítését írja le részletesen, az eredmények fejezet pedig táblázatokkal és ábrákkal mutatja be, hogyan teljesítettek a modellek. A végén felhasználói és telepítési útmutató segíti a rendszer üzembe helyezését, az összefoglalás pedig a tanulságokat és a lehetséges következő lépéseket foglalja össze.

1. Adatok és módszertan

1.1 Adatok bemutatása

Az adatok Dr. Tiszai Luca kutatási programjához kötődnek [2], aki azt vizsgálja, hogy a halmozottan fogyatékos személyek zenei élményei hogyan mérhetők objektív eszközökkel. A gyűjtés során halmozottan fogyatékos és egészséges személyek EEG-jeleit rögzítették zenehallgatás közben, és a kérdés az volt, hogy az agyi aktivitásban megjelennek-e olyan mintázatok, amelyek alapján a két csoport szétválasztható egymástól.

Mielőtt ezekkel az adatokkal elkezdtem volna dolgozni, egy korábbi fázisban Parkinson-kóros betegek EEG-felvételein [1] próbáltam ki a különböző megközelítéseket. Ezek .set kiterjesztésű fájlokban érkeztek, ami az EEGLAB neurológiai elemzőeszköz saját formátuma, nyers EEG idősorokat tartalmaz csatornainformációkkal és eseményjelölőkkel együtt. Itt tanultam meg rendesen használni a Pythont és a releváns könyvtárakat, főleg a numpy-t, a pandast és a scikit-learn-t. Konkrét eredmény nem lett belőle, de a tapasztalat, amit szereztem, később nagyon jól jött.

A tényleges adatállomány 60 felvételt tartalmaz CSV formátumban, pontosvesszővel elválasztva. Ebből 26 halmozottan fogyatékos és 25 egészséges személy felvétele ment bele a tanításba, a maradék 9-et pedig félretettem manuális tesztelésre. Ezeket a modellt sosem látta tanítás közben, így a grafikus felületen valódi független tesztesetekként lehet őket használni, ami őszintébb képet ad a rendszer tényleges teljesítményéről.

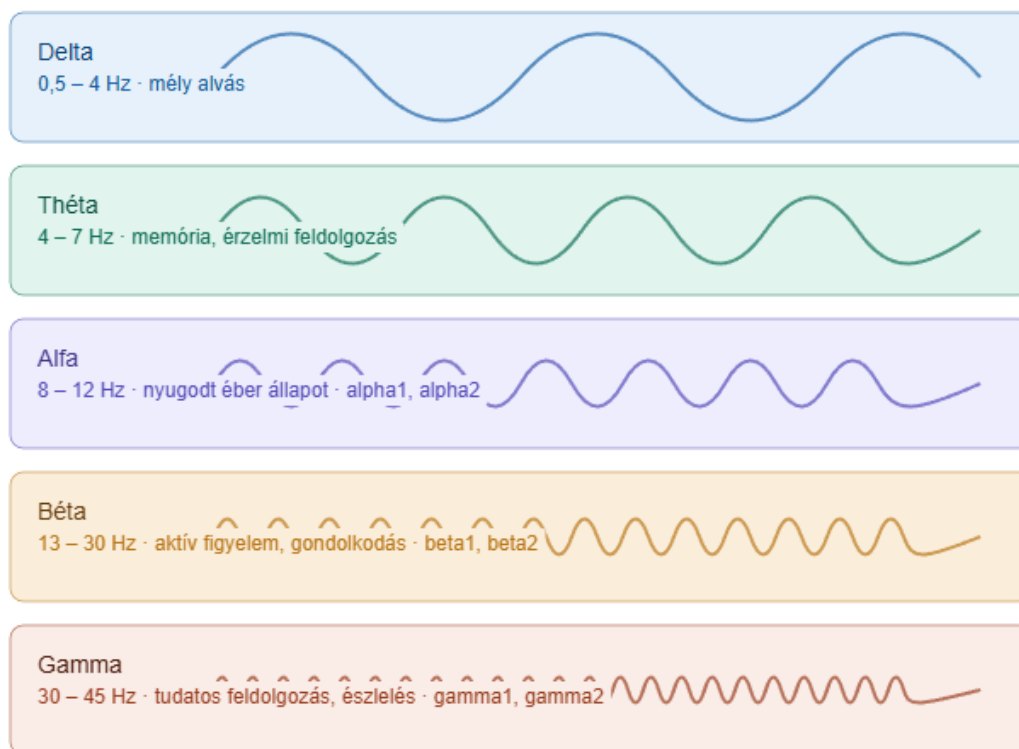
Minden fájl ugyanolyan szerkezetű, soronként egy időpillanat adatai szerepelnek, összesen 13 oszlopban: time, rawEEG, attention, meditation, signalStrength, alpha1, alpha2, beta1, beta2, delta, gamma1, gamma2, theta, és blink, ahol ez utóbbi a pislogásokat jelöli. Nem minden oszlop ugyanolyan típusú adatot tartalmaz: a rawEEG a nyers jelet tárolja milliszekundumos felbontásban, a frekvenciasáv oszlopok viszont már előfeldolgozott értékek, amelyeket az eszköz a Fourier-transzformáció után számol, és ritkábban is frissülnek. Nem használtam fel az összes oszlopot, a rawEEG, a delta, a signalStrength és a blink végül kimaradt, mert osztályozás szempontjából nem bizonyultak hasznosnak. Ami megmaradt: alpha1 és alpha2 az alfa tartomány két alsávjaként, beta1 és beta2 a béta tartományból, theta, gamma1 és gamma2 a gamma

tartományból, valamint az attention és meditation oszlopok, amelyek az eszköz által becsült figyelmi és meditációs szinteket tartalmazzák.

1.2 Az EEG és frekvenciasávok

Az EEG, vagyis az elektroencefalográfia lényege az, hogy az agy elektromos aktivitását kívülről, non-invazív módon méri az agykéreg közelében elhelyezett elektródák segítségével. A neuronok által kibocsátott elektromos jelek összeadódnak, és a fejbőrön keresztül is mérhetővé válnak. Más képalkotó eljárásokhoz képest, például az MRI-hez, az EEG időbeli felbontása kiemelkedő: milliszekundumos pontossággal követi az aktivitás változásait, ami zenei feldolgozás vagy kognitív reakciók vizsgálatánál kifejezetten sokat számít. Ez teszi alkalmassá arra, hogy olyan gyors agyi folyamatokat is nyomon lehessen vele követni, amelyek más módszerekkel egyszerűen nem ragadhatók meg megfelelő pontossággal.

A delta sáv (0,5–4 Hz) a leglassabb agyhullámokat tartalmazza, és elsősorban mély alvás közben jelenik meg. Éber állapotban rögzített felvételekben szinte alig szerepel, ezért nem bizonyult informatív jellemzőnek a kutatás szempontjából, és végül kihagytam a feldolgozásból. A theta sáv (4–7 Hz) féléber állapothoz, memóriafolyamatokhoz és érzelmi feldolgozáshoz kapcsolódik. Zenehallgatásnál ez kimondottan érdekes lehet, hiszen a zene érzelmi hatása és a memória aktiválása nagyrészt ebben a sávban jelenik meg, így ez az egyik legfontosabb jellemző a vizsgált feladat szempontjából. Az alfa sáv (8–12 Hz) a nyugodt, relaxált éber állapot jellemzője, és általában csökken, amikor valaki aktívan figyel valamire vagy mentálisan leterhelt. Az adatokban ez a sáv két alsávra van bontva, alpha1-re (8–10 Hz) és alpha2-re (10–12 Hz), ami finomabb különbségek megragadását teszi lehetővé a két csoport között. A béta sáv (13–30 Hz) aktív gondolkodáshoz, koncentrációhoz és éber figyelemhez kötődik, és szintén két részre van osztva, beta1-re (13–20 Hz) és beta2-re (20–30 Hz). A gamma sáv (30–45 Hz) a legmagasabb frekvenciájú hullámokat foglalja magában, és magasabb szintű kognitív folyamatokhoz kapcsolódik, például észleléshez, figyelemhez és tudatos feldolgozáshoz. Ez is két alsávra van bontva az adatokban, gamma1-re (30–40 Hz) és gamma2-re (40–45 Hz), ami szintén finomabb elemzést tesz lehetővé. [6]



A hullámok periódushossza arányos a valós frekvenciával – minél gyorsabb a rezgés, annál magasabb a frekvencia

1.1 ábra: Az EEG öt fő frekvenciasávja [7]

Az attention és meditation értékek nem hagyományos frekvenciasávok, hanem a rögzítő eszköz belső algoritmusai számítják őket a nyers jelből. Az attention a koncentráció szintjét becsüli meg, a meditation a mentális nyugalomét. Nem közvetlenül mért fizikai mennyiségek, hanem számított mutatók, viszont úgy döntöttem, beleveszem őket a modellbe, mert szerintem hasznos kiegészítő információt hordozhatnak az osztályozás szempontjából, még ha közvetett módon is.

1.3 Jellemzőkinyerési módszerek

A gépi tanulási modellek nem tudnak közvetlenül nyers idősorokkal dolgozni, ezért minden EEG-felvételből egy fix méretű számvektort kell előállítani, amely tömören leírja a felvétel legfontosabb tulajdonságait. Ezt a folyamatot jellemzőkinyerésnek nevezzük. A rendszerben három ilyen módszer került kidolgozásra, amelyek különböző megközelítésekkel írják le ugyanazokat az adatokat, és amelyek teljesítménye a tanítás során egymással is összehasonlításra kerül.

Az első módszer az `extract_statistical`, amely statisztikai jellemzőkre épül. Minden felhasznált csatornára, vagyis az `alpha1`, `alpha2`, `beta1`, `beta2`, `theta`, `gamma1`, `gamma2`,

attention és meditation oszlopokra három alapstatisztikát számol: az átlagot, a szórást és a teljesítménybecslést, ami az átlagos négyzetes értéknek felel meg. Az átlag megmutatja, hogy az adott sáv teljesítménye átlagosan milyen szinten mozog a felvétel során, a szórás azt jelzi, mennyire változó ez időben, a teljesítménybecslés pedig a jel energiájának egy egyszerű közelítése. A 9 csatorna és 3 statisztika összesen 27 jellemzőt eredményez. Ezen felül a módszer hat frekvenciasáv-arányt is kiszámol: az α_1/β_1 és α_2/β_2 arányokat, a θ/α_1 és θ/α_2 arányokat, valamint a γ_1 és γ_2 részesedését az összes sáv teljesítményéből. Ezek az arányok azért hasznosak, mert nem az abszolút értékek, hanem a sávok egymáshoz viszonyított viszonya hordoz fontos információt, ami személyenként stabilabb jellemzőnek bizonyul. Az `extract_statistical` összesen 33 jellemzőt ad vissza egyetlen felvételtől.

A második módszer az `extract_phase_epoch`, amely a fázis spektrum elemzésére épül. A felvételt egyenlő hosszú, körülbelül 10 másodperces epochokra osztja fel, maximum 5 epochot dolgoz fel, majd minden epochra és csatornára elvégzi a gyors Fourier-transzformációt. Ennek eredménye komplex számok sorozata, amelyből a `np.angle()` függvény segítségével kinyerhető a fázis spektrum, ami megmutatja, hogy az egyes frekvencia-összetevők milyen szögben, vagyis milyen időbeli eltolással vannak jelen a jelben. Minden epochból és csatornából három fázis-jellemző kerül kinyerésre: a fázisszögek átlaga, szórása és maximuma. Az epocholás azért előnyös, mert a felvétel különböző időszakait külön-külön is figyelembe veszi, így az időbeli változások is megjelennek a jellemzővektorban.

A harmadik módszer az `extract_phase_band`, amely az előző továbbfejlesztett változata. Ugyanúgy epochokra bontja a felvételt és Fourier-transzformációt végez, de a fázis spektrumból nem általánosan, hanem frekvenciasávonként nyeri ki a jellemzőket. Minden epochban és csatornában külön meghatározza a fázisszögek átlagát az α_1 (8–10 Hz), α_2 (10–12 Hz), β_1 (13–20 Hz), β_2 (20–30 Hz), θ (4–7 Hz), γ_1 (30–40 Hz) és γ_2 (40–45 Hz) sávokban. Ez finomabb képet ad arról, hogy az egyes frekvenciatartományokban hogyan viselkedik a jel fázisa, ami neurológiai szempontból is informatívabb lehet, hiszen az egyes sávok eltérő kognitív folyamatokhoz köthetők. Mindhárom módszernél fontos követelmény, hogy a visszaadott jellemzővektor mindig azonos méretű legyen, függetlenül a felvétel hosszától. Ha egy felvétel túl rövid ahhoz, hogy az összes epochot kitöltse, a hiányzó epochok helyére nullák kerülnek, így a modellek minden esetben egyforma méretű bemenetet kapnak.

1.4 Gépi tanulási algoritmusok

A jellemzőkinyerés után a kapott vektorokat gépi tanulási modellek segítségével osztályozom. Négy modellt alkalmazok, amelyek eltérő elveken működnek.

Az SVM célja, hogy megtalálja azt a határfelületet, amely a legjobban szétválasztja a halmazottan fogyatékos és egészséges személyek felvételeit egymástól. Nem akármilyen határfelületet keres, hanem azt, amelyik a két osztály legközelebbi pontjaitól a legtávolabb van, vagyis a legnagyobb margót biztosítja a két csoport között. Ha az adatok nem választhatók szét lineárisan, kernel-függvényekkel magasabb dimenziós térbe vetíthetők, ahol a szétválasztás már elvégezhető. A leggyakrabban alkalmazott kernelek a radiális bázisfüggvény, a polinomiális és a szigmoid kernel, amelyek közül mindhárommal kísérleteztem. [8]

A Random Forest sok egymástól független döntési fát épít fel, és ezek szavazása alapján dönti el, melyik osztályba tartozik egy minta. Minden fa az adatok egy véletlenszerű részhalmazán tanul, ami csökkenti a túlilleszkedés kockázatát és robusztussá teszi a modellt a zajjal szemben. Nagy dimenziójú adatokkal jól boldogul, kevés előfeldolgozást igényel, és általában stabil eredményeket ad, viszont nagyobb adathalmazon lassabb lehet más módszereknél. [9]

Az XGBoost egymás után épít fel döntési fákat, és minden lépésben az előző modell hibáit próbálja javítani. Táblázatos adatokon általában jó eredményeket ér el, regularizációt is alkalmaz a túlilleszkedés ellen, és jól kezeli a hiányzó értékeket. Versenyek és gyakorlati tapasztalatok alapján az egyik legmegbízhatóbb osztályozónak számít, különösen kisebb adathalmazok esetén. [10]

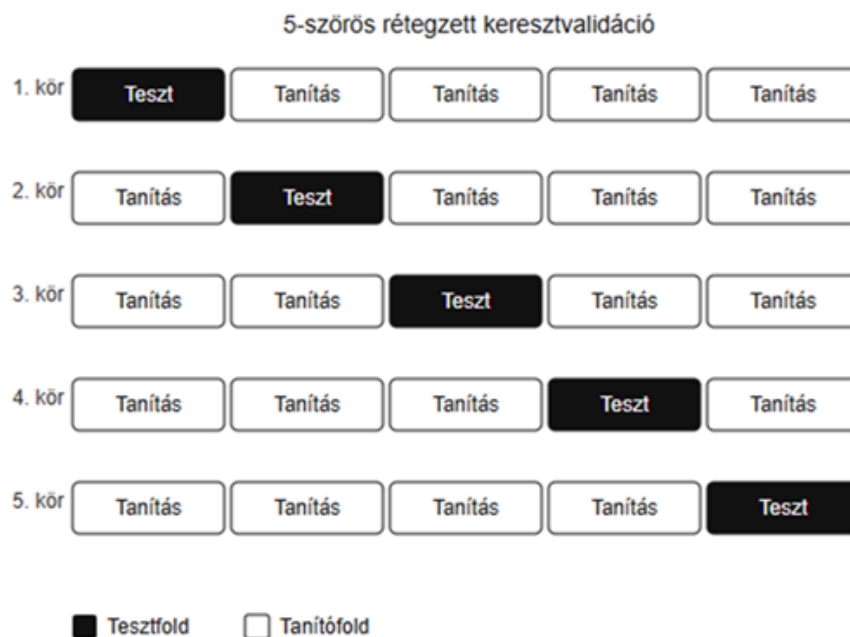
A negyedik modell egy Random Forest és egy neurális háló kombinációja. A Random Forest először valószínűségi értékeket állít elő arról, hogy egy minta mekkora eséllyel tartozik az egyes osztályokba, majd ezeket kapja meg bemenetként a neurális háló, amely meghozza a végső döntést. Ez a kétlépéses megközelítés ötvözi a Random Forest robusztusságát a neurális hálók nemlineáris tanulási képességével, és olyan mintázatok felismerésére is alkalmas lehet, amelyeket egyik módszer önmagában nem feltétlenül venne észre.

1.5 Hiperparaméter-optimalizálás és keresztvalidáció

A modellek teljesítménye nagyban függ a hiperparamétereiktől, amelyek olyan beállítások, amelyeket nem a tanítás során tanul meg a modell, hanem előre kell meghatározni. Ilyen például az SVM esetén a C együttható és a kernel típusa, a Random Forest esetén a fák száma és mélysége, az XGBoost esetén pedig a tanulási ráta. Ezek megválasztása komolyan befolyásolja, hogy a modell mennyire teljesít jól olyan adatokon, amelyeket korábban nem látott.

A hiperparaméter-optimalizálásra GridSearch módszert alkalmazok. Minden paraméterhez megadok egy listát a lehetséges értékekkel, és az algoritmus az összes kombinációt végigpróbálja, majd kiválasztja a legjobban teljesítőt. Számításigényes megoldás, de kis adathalmazon jól működik és megbízható eredményt ad.

A modellek kiértékelésére 5-szörös rétegzett keresztvalidációt használok. Az adatokat 5 részre osztom, és 5 körben értékelem ki a modellt: minden körben más rész kerül tesztelésre, a többi pedig tanításra szolgál. Az 5 kör átlagolt eredménye megbízhatóbb képet ad a valódi teljesítményről, mint egyetlen train/test felosztás alapján kapnék. A rétegzett változatot azért választottam, mert ez biztosítja, hogy minden körben arányosan legyenek jelen a beteg és egészséges minták, ami kisebb és enyhén kiegyensúlyozatlan adathalmazok esetén különösen fontos. A teljesítményt pontossággal, kiegyensúlyozott pontossággal és konfúziós mátrixszal értékelem, amelyek együttesen átfogó képet adnak arról, hogyan teljesít a rendszer az osztályozási feladaton.



1.2. ábra: Az 5-szörös rétegzett keresztvalidáció működése [11]

1.6 Kiértékelési metrikák

A gépi tanulási modellek kiértékelésekor egyetlen metrika önmagában sokszor nem ad teljes képet, különösen kisebb vagy enyhén kiegyensúlyozatlan adathalmazok esetén. Ezért minden modellnél egyszerre három mutatót számolok ki, amelyek más-más oldalról világítják meg, hogyan teljesít az osztályozó.

A pontosság a legkézenfekvőbb metrika, azt mutatja meg, hogy az összes mintából hányat osztályozott helyesen a modell. Hátránya viszont, hogy kiegyensúlyozatlan adatokon félrevezető lehet. Ha például az adatok 90%-a egészséges személy felvétele, egy olyan modell is 90%-os pontosságot érhet el, amely minden mintát egészségesnek oszt be, és valójában semmit sem tanult meg. Ez a helyzet a jelen kutatásban ugyan nem áll fenn ilyen mértékben, de az adathalmaz mérete miatt mégis érdemes óvatosan kezelni ezt a mutatót.

Erre ad megoldást a kiegyensúlyozott pontosság, amely nem az összes minta, hanem az egyes osztályokon belüli helyes osztályozási arányok átlagát számítja. A beteg és az egészséges csoport teljesítménye így egyforma súllyal szerepel a végeredményben, és nem fordulhat elő, hogy az egyik osztály dominanciája eltorzítja a képet. Az 51

tanítóadatokból álló halmazon ez sokkal őszintébb visszajelzést ad a modell valódi teljesítményéről.

A konfúziós mátrix nem egyetlen számot, hanem egy táblázatot ad vissza, amelyből pontosan látszik, hogy a modell melyik osztályban tévedett és mennyit. A négy cella a valódi pozitív, valódi negatív, hamis pozitív és hamis negatív eseteket tartalmazza, amiből kiderül például, hogy a modell inkább a beteg vagy az egészséges mintákat téveszti össze. Ez diagnosztikai szempontból különösen hasznos információ. Mindhárom metrika az 5-szörös keresztvalidáció minden körében kiszámolódik, és a végső érték az 5 kör átlagaként jelenik meg.

2. Implementáció

2.1 Projekt felépítése

A rendszer egyetlen mappában helyezkedik el, ahol az adatok és a forráskód fájlok egymás mellett vannak. Az adatokat a data mappa tárolja, amelyen belül három almappa található: a beteg mappa a halmozottan fogyatékos személyek felvételeit tartalmazza, az egészséges az egészséges személyekét, a test mappa pedig azokat a felvételeket, amelyeket kizárólag manuális tesztelésre tartottam fenn. Ezeket a modellt a tanítás során sosem látta, így a grafikus felületen valódi független tesztesetekként funkcionálnak.

A forráskód öt Python fájlból áll. A `feature_extraction.py` tartalmazza a jellemzőkinyerő függvényeket, tehát az `extract_statistical()`, `extract_phase_epoch()` és `extract_phase_band()` metódusokat. A `train_model.py` felelős a modellek betanításáért és kiértékeléséért, itt van megvalósítva a keresztvalidáció és a jellemzők fontosságának kinyerése is. A `gui.py` valósítja meg a grafikus felületet, amely a jellemzőkinyerést és a modelleket egy könnyen kezelhető alkalmazásba köti össze. Az `epoch_analysis.py` egy önállóan futtatható szkript, amely epochonként hasonlítja össze a beteg és egészséges csoportokat. A `feature_selection.py` szintén önállóan fut, és azt vizsgálja, hogy a legfontosabb N jellemző használata javít-e a modellek teljesítményén.

A betanított modellek `.pkl` fájlkként mentődnek a projekt gyökérmappájába, ezek az `svm_model.pkl`, `rf_model.pkl`, `rf_mlp_model.pkl` és `xgb_model.pkl`. A GUI ezeket tölti be, amikor új felvételt kell osztályozni, tehát a modelleket elegendő egyszer betanítani, és utána a grafikus felület közvetlenül ezekből dolgozik.

A projekt két további önállóan futtatható elemző szkriptet is tartalmaz. Az `epoch_analysis.py` a beteg és egészséges csoport EEG-jeleit hasonlítja össze epochonként, min-max kontraszt elemzéssel és kétmintás t-tesztel, és az eredményeket grafikonon is megjeleníti. A `feature_selection.py` azt vizsgálja, hogy a modellek teljesítménye javítható-e azzal, ha csak a legfontosabb 10, 15 vagy 20 jellemzőt használjuk a tanítás során. A `visualization.py` három vizualizációt készít: egy 2D szórásdiagramot a két legfontosabb jellemző alapján, egy pókháló diagramot mind a 33 jellemzőre, valamint a Random Forest modell egyik döntési fájának rajzát. Ez utóbbi

három szkript a GUI-tól teljesen függetlenül működik, és közvetlenül PyCharmból vagy parancssorból futtatható.



2.1 ábra: A projekt fájlnak és mappáinak elrendezése

2.2 Jellemzőkinyerés – `feature_extraction.py`

A `feature_extraction.py` fájl tartalmazza a három jellemzőkinyerő függvényt, amelyek mindegyike egy CSV fájl elérési útját kapja bemenetként, és egy numpy tömböt ad vissza. Ez a tömb az adott felvétel jellemzővektora, amelyet a modellek a tanítás és az osztályozás során felhasználnak. A három függvény között az a közös, hogy mindegyik ugyan azokból az oszlopokból dolgozik, és mindegyik ugyanazokkal a statisztikákkal és sávarányokkal egészíti ki a jellemzővektort a végén. A különbség abban rejlik, hogy a két FFT-alapú függvény ezen felül időbeli bontást is alkalmaz, és a fázis spektrumból is kinyeri a jellemzőket.

Az első függvény az `extract_statistical()`, amely kizárólag statisztikai jellemzőkre épül. Beolvassa a CSV fájlt, majd minden felhasznált csatornára: `alpha1`, `alpha2`, `beta1`, `beta2`, `theta`, `gamma1`, `gamma2`, `attention`, `meditation` – kiszámol három alapstatisztikát: az átlagot, a szórást és a teljesítménybecslést, amely az átlagos négyzetes értéknek felel meg. Az átlag megmutatja, hogy az adott sáv teljesítménye átlagosan

milyen szinten mozog a felvétel során, a szórás azt jelzi, mennyire ingadozik ez az érték időben, a teljesítménybecslés pedig a jel energiájának egyszerű közelítése. A kilenc csatorna és három statisztika összesen 27 jellemzőt ad. Ezen felül hat frekvenciasáv-arány is kiszámolódik: az α_1/β_1 és α_2/β_2 hányadosok, a θ/α_1 és θ/α_2 hányadosok, valamint a γ_1 és γ_2 részesedése az összes sáv teljesítményéből. Ezek az arányok azért hasznosak, mert a sávok egymáshoz viszonyított viszonya személyenként stabilabb jellemzőnek bizonyul, mint az abszolút értékek. Az osztásoknál minden nevezőhöz egy kis $1e-9$ értéket ad hozzá, ami nullával való osztást akadályoz meg. Az `extract_statistical()` összesen 33 jellemzőt ad vissza egyetlen felvételtől.

A második függvény az `extract_phase_epoch()`, amely már időbeli bontást is alkalmaz. A felvételt, amely jellemzően 30-40 ezer sort tartalmaz, egyenlő hosszú, körülbelül 10 másodperces epochokra osztja fel, és maximum 5 epochot dolgoz fel. Az epochhossz 512 sor, ami a rögzítés mintavételezési frekvenciájából (időbélyeg alapján kb. 50 Hz) adódóan körülbelül 10 másodpercnek felel meg. Minden epochra és minden csatornára elvégzi a gyors Fourier-transzformációt, majd az `np.angle()` függvénnyel kinyeri a fázis spektrumot, ami megmutatja, hogy az egyes frekvencia-összetevők milyen szögben, vagyis milyen időbeli eltolással vannak jelen a jelben:

```
fft_vals = np.fft.rfft(epoch[:, j])
phase = np.angle(fft_vals)
features.extend([np.mean(phase), np.std(phase),
np.max(np.abs(phase))])
```

Minden epochból és csatornából tehát három fázis-jellemző kerül kinyerésre: a fázisszögek átlaga, szórása és maximuma. Az epocholás azért előnyös, mert a felvétel különböző időszakaszait külön-külön is figyelembe veszi, így az időbeli változások is megjelennek a jellemzővektorban. Ha a felvétel rövidebb mint 5 epoch, a hiányzó epochok helyére nullák kerülnek, hogy a jellemzővektor mindig egyforma méretű legyen. A függvény az epochos fázis jellemzők mellett ugyanazokat a statisztikákat és sávarányokat is tartalmazza mint az `extract_statistical()`.

A harmadik függvény az `extract_phase_band()`, amely az előző továbbfejlesztett változata. Ugyanúgy epochokra bontja a felvételt és Fourier-transzformációt végez, de a fázis spektrumból nem általánosan, hanem

frekvenciasávonként nyeri ki a jellemzőket. Ehhez előre meghatározza az egyes sávok frekvenciaindex-tartományait az ~50 Hz-es mintavételezési frekvencia alapján:

```
freqs = np.fft.rfftfreq(epoch_len, 1/sfreq)
alpha1_idx = (freqs >= 8) & (freqs <= 10)
alpha2_idx = (freqs >= 10) & (freqs <= 12)
beta1_idx = (freqs >= 13) & (freqs <= 20)
beta2_idx = (freqs >= 20) & (freqs <= 30)
theta_idx = (freqs >= 4) & (freqs <= 7)
gamma1_idx = (freqs >= 30) & (freqs <= 40)
gamma2_idx = (freqs >= 40) & (freqs <= 45)
```

Majd minden epochban és csatornában külön kiszámítja az egyes sávokhoz tartozó fázisszögek átlagát. A `np.any(band_idx)` feltétel biztosítja, hogy üres sávindex esetén se keletkezzen hiba, hanem nullával töltődjön fel az adott pozíció. Ez a megközelítés finomabb képet ad arról, hogy az egyes frekvenciatartományokban hogyan viselkedik a jel fázisa, ami neurológiai szempontból is informatívabb lehet, hiszen az egyes sávok eltérő kognitív folyamatokhoz köthetők. Az `extract_phase_band()` a három módszer közül a legtöbb jellemzőt adja vissza, mivel epochonként és csatornánként mind a hét sávra külön fázisátlagot számol.

Mindhárom függvény esetében fontos megjegyezni, hogy az adatokban szereplő frekvenciasáv oszlopok: `alpha1`, `alpha2` stb, már előfeldolgozott sávteljesítmény értékek, nem nyers EEG idősorok. Az FFT tehát ezeken az előfeldolgozott értékeken fut le, ami nem ugyanaz mint a nyers jelre alkalmazott Fourier-transzformáció, ezt a 2.5 fejezetben tárgyalt epoch analízis eredményei is figyelembe veszik.

2.3 Modellek betanítása

A `train_model.py` fájl tartalmazza a négy gépi tanulási modell betanításához és kiértékeléséhez szükséges függvényeket, valamint egy közös segédfüggvényt a keresztvalidációhoz. Minden betanító függvény ugyanazt a struktúrát követi: először `GridSearch` segítségével megkeresi a legjobb hiperparamétereket, majd keresztvalidációval kiértékeli a modellt, végül a teljes adaton betanítja és elmenti.

A közös `kfold_evaluate()` függvény végzi az 5-szörös rétegzett keresztvalidációt. Az adatokat 5 egyenlő részre osztja úgy, hogy minden részben

arányosan szerepeljenek a beteg és egészséges minták, majd minden körben más rész kerül tesztelésre:

```
skf = StratifiedKFold(n_splits=N_SPLITS,
shuffle=True, random_state=42)
for train_idx, test_idx in skf.split(X, y):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

Az 5 kör pontossági és balanced accuracy értékeit átlagolja, a konfúziós mátrixokat pedig összeadja és átlagolja. Ez megbízhatóbb képet ad a modell valódi teljesítményéről, mint egyetlen train/test split.

Az első betanító függvény a `train_and_evaluate_svm()`, amely egy Support Vector Machine modellt tanít be. A GridSearch az SVM három legfontosabb hiperparaméterét: a C regularizációs együtthatót, a gamma értéket és a kernel típusát optimalizálja, az összes lehetséges kombinációt kipróbálva. Mivel az SVM-nek nincs beépített feature importance mechanizmusa, a jellemzők fontosságát permutation importance segítségével számítja ki, ez 10-szer újravégrehajtja a modellt, minden alkalommal egy jellemző értékeit felkeverve, és méri mennyit romlott a pontosság:

```
perm = permutation_importance(best_model, X, y,
n_repeats=10, random_state=42, n_jobs=-1)
feature_importances = perm.importances_mean
```

A `train_and_evaluate_rf()` a Random Forest modellt tanítja be. A GridSearch a fák számát, maximális mélységét és az elágazási feltételek minimális mintaszámát optimalizálja. A Random Forest esetén a feature importance beépítetten elérhető a betanított modellből, nem kell külön számítani:

```
feature_importances = best_model.feature_importances_
```

A `train_and_evaluate_rf_mlp` egy összetettebb modellt valósít meg, amelyet az `RF_MLP_Combined` osztály reprezentál. Ez az osztály a `BaseEstimator` és `ClassifierMixin` scikit-learn alaposztályokból örökli az interfészt, ami lehetővé teszi hogy a keresztvalidációs ciklusban ugyanúgy lehessen használni mint a többi modellt. A

működés lényege hogy a Random Forest valószínűségi kimeneteit adja át egy neurális hálónak:

```
def fit(self, X, y):  
    self.rf_model.fit(X, y)  
    rf_proba = self.rf_model.predict_proba(X)  
    self.mlp_model.fit(rf_proba, y)  
    return self
```

Az MLP két rejtett réteggel rendelkezik: 32 és 16 neuronnal, ReLU aktivációs függvényrel és Adam optimalizálóval. A feature importance itt a Random Forest részéből kerül kinyerésre.

A `train_and_evaluate_xgb` az XGBoost modellt tanítja be, amelynek GridSearch-e a legtöbb paramétert vizsgálja: a fák számát, a tanulási rátát, a fa mélységét, a mintavételezési arányt és az oszlopok mintavételezési arányát. Az XGBoost-nál is beépítetten elérhető a feature importance. Mindegyik függvény végén a betanított modell .pkl fájlba mentődik a joblib könyvtár segítségével, amit a GUI később betölt osztályozáskor.

2.4 Epoch analízis – `epoch_analysis.py`

Az `epoch_analysis.py` egy önállóan futtatható szkript, amely a GUI-tól teljesen függetlenül működik. Célja, hogy epochonként összehasonlítsa a beteg és egészséges csoport EEG-jeleit, és feltárja az esetleges különbségeket. A szkript két elemzést végez: egy kontraszt elemzést és egy kétmintás t-tesztet.

A fájl elején globális konfigurációs változók találhatók, amelyek meghatározzák az adatok helyét, a mintavételezési frekvenciát és az epochok hosszát. Ez azért hasznos, mert ha ezeket az értékeket meg kell változtatni, elegendő egy helyen módosítani.

Az adatok betöltését a `load_epochs_from_dir()` függvény végzi, amely beolvassa az összes CSV fájlt egy mappából, és 512 soros epochokra bontja őket, ugyanolyan logikával mint a `feature_extraction.py`-ban. A függvény visszatérési értéke egy lista, amelynek minden eleme egy szótár: a csatorna neve a kulcs, az epochok listája az érték.

A kontraszt elemzést a `compute_contrast()` függvény végzi. Epochonként és csatornánként kiszámítja a minimum és maximum értékek közötti különbséget minden felvételre, majd ezeket összegyűjti:

```
c = np.max(epoch) - np.min(epoch)
contrast[col][epoch_idx].append(c)
```

Ez a mutató megmutatja, mekkora az amplitúdó-ingadozás az adott epochban. Minél nagyobb ez az érték, annál változékonyabb a jel az adott időszakban.

A szignifikancia vizsgálatot a `compute_ttest()` függvény végzi kétmintás t-teszt segítségével, amely epochonként és csatornánként összehasonlítja a beteg és egészséges csoport kontrasztértékeit. A t-teszt megmondja, hogy a két csoport között mért különbség statisztikailag szignifikáns-e, vagy csupán véletlenszerű ingadozásból adódik – ha a p-érték 0.05 alatt van, a különbség szignifikánsnak tekinthető. A teszt legalább 2 mintát igényel mindkét csoportból, ha ez nem teljesül, a függvény p=1 értéket ad vissza, ami nem szignifikáns eredményt jelent.

Az eredmények megjelenítését a `plot_all()` függvény végzi, amely egyetlen ábrán belül két sorba rendezi a grafikonokat, a felső sorban a kontraszt, az alsó sorban a t-teszt p-értékei láthatók, minden csatornára külön oszlopdiagramon. A sötét oszlopok a szignifikáns különbségeket jelzik ($p < 0.05$), a világosak a nem szignifikánsakat, és egy piros szaggatott vonal jelöli a 0.05-ös határt. A `print_summary()` függvény ugyanezeket az eredményeket szöveges formában írja a konzolra, ami megkönnyíti a konkrét számok leolvasását.

2.5 Jellemzőkiválasztás – `feature_selection.py`

A `feature_selection.py` azt vizsgálja, hogy érdemes-e visszafogni a jellemzők számát, és ha igen, mennyi az a pont, ahol a modell már nem teljesít jobban, hanem rosszabbul. A gépi tanulásban ismert jelenség, hogy felesleges vagy zajt hordozó jellemzők ronthatják az általánosító képességet, kis adathalmazon pedig különösen veszélyes, ha túl sok jellemzővel dolgozunk, mert ilyenkor könnyen túlilleszkedik a modell a tanítóadatokra.

Mindhárom jellemzőkinyerési módszerre lefuttatom az összes modellt, először az összes jellemzővel, majd a feature importance alapján kiválasztott top 10, top 15 és top 20 legfontosabb jellemzővel. A fontossági sorrendet ugyanúgy határozom meg, mint a

train_model.py-ban: Random Forest és XGBoost esetén a beépített feature_importances_ tulajdonsággal, SVM esetén permutation importance segítségével. A kiválasztást a select_top_features() függvény végzi, amely importancia szerint rendezi a jellemzőket és kiveszi a legfontosabbakat.

Az eredmények a konzolra íródnak ki, és minden modellnél és jellemzőkinyerési módszernél látható az összes jellemzővel elért pontosság, mellette a top 10, 15 és 20 jellemzővel kapott érték az eltérés előjelével együtt. Ebből egyből kiderül, hogy egy adott konfiguráció esetén a jellemzők szűkítése segít, ront, vagy nem változtat érdemben a teljesítményen. A részletes számok az eredmények fejezetben szerepelnek.

2.6 Vizualizáció – visualization.py

A visualization.py három vizualizációt készít az adatokból és a betanított modellekből. Az egész szkript lényege, hogy ne csak számok legyenek az eredmények, hanem vizuálisan is látható legyen, hogyan különülnek el a csoportok és hogyan gondolkodik a modell.

Az adatokat a load_data() függvény tölti be, amely az extract_statistical() függvényt használja, tehát ugyanazokat a 33 jellemzőt dolgozza fel mint a tanítás során. A jellemzők neveit a get_feature_names() adja vissza, ezeket használom feliratként a grafikonokon.

Az első ábra egy 2D szórásdiagram, amelyet a plot_scatter() függvény készít. Két plot jelenik meg egymás mellett: a bal oldali az SVM két legfontosabb jellemzőjét, a beta2_mean-t és az alpha2_std-t ábrázolja, a jobb oldali az RF+MLP két legfontosabb jellemzőjét, a beta1_std-t és az alpha2_std-t. Minden pont egy felvételt jelöl, piros a beteg, zöld az egészséges. Az ábra lényege, hogy megmutassa, mennyire válnak szét a csoportok, ha csak a két legmértékesebb jellemzőt nézzük.

A második egy pókháló diagram, amelyet a plot_radar() készít, és mind a 33 jellemzőt egyszerre ábrázolja. Minden tengelyen az adott jellemző normalizált átlagértéke szerepel, a normalizálás azért kell, mert a jellemzők eltérő skálán mozognak. A piros vonal a beteg, a zöld az egészséges csoport átlagát mutatja, és egy pillantással látszik, hol különböznek egymástól a legjobban.

A harmadik egy döntési fa ábra, amelyet a plot_decision_tree() készít. A már elmentett Random Forest modell egyik fáját rajzolja ki, de csak 3 szint mélységig, mert ennél mélyebbre menve olvashatatlaná válna. Megmutatja, hogy a modell melyik

jellemző alapján oszt el először, és az egyes ágakon milyen küszöbértékek mentén jut el a döntésig. Ez azért hasznos, mert láthatóvá teszi, hogy a modell nem véletlenszerűen osztályoz, hanem konkrét szabályok mentén.

2.7 Grafikus felhasználói felület – gui.py

A gui.py fájl valósítja meg a grafikus felhasználói felületet, amely a tkinter könyvtárra épül. A felület összekötő kapcsolatot jelent a jellemzőkinyerés, a modellek betanítása és a predikció között, a felhasználónak nem kell közvetlenül a kóddal dolgoznia. A fájl importálja a feature_extraction.py és a train_model.py modulokat, és ezek függvényeit köti össze a grafikus elemekkel.

A felületen két legördülő menü és hat gomb található. Az első menüben a modellt lehet kiválasztani, SVM, Random Forest, Random Forest MLP vagy XGBoost közül, a másodikban a jellemzőkinyerési módszert, Statistical, Phase epoch vagy Phase band közül. A modellek és a hozzájuk tartozó jellemzőkinyerési függvények szótárakban vannak tárolva. Ez azért hasznos, mert ha később új modellt vagy módszert kell berakni, csak a szótárat kell bővíteni, minden más változatlan marad.

A hat gomb mindegyike mást csinál. A Tanítás gomb elindítja a betanítást a kiválasztott modellel és jellemzőkinyerési módszerrel, az eredmény globális változókba kerül. A Vizualizáció gomb egy PCA-alapú szórásdiagramot dob fel, amelyen látható, hogy a beteg és egészséges minták hogyan különülnek el egymástól a jellemzőtérben. A Tanítás adatai gomb egy külön ablakban mutatja meg a pontosságot, a balanced accuracy-t és a konfúziós mátrixot. A Jellemzők fontossága gomb egy vízszintes oszlopdiagramon mutatja meg a top 20 legfontosabb jellemzőt. A Fájl feltöltése és predikció gomb arra való, hogy be lehessen tölteni egy új EEG CSV fájlt, amelyet a már betanított modell azonnal osztályoz és kiírja az eredményt.

A program pontos kezelési útmutatója az 4. fejezetben van, ahol lépésről lépésre le van írva, hogyan kell használni.

3. Eredmények

3.1 A modellek teljesítményének összehasonlítása

A rendszer négy gépi tanulási modellt: SVM, Random Forest, Random Forest MLP, XGBoost és három jellemzőkinyerési módszert: Statistical, Phase epoch és Phase band tartalmaz. A modellek kiértékelése 5-szörös rétegzett keresztvalidációval történt, ahol minden körben az adatok 1/5-e kerül tesztelésre, a maradék 4/5-e tanításra. A végső pontosság és balanced accuracy az 5 kör átlaga. A konfúziós mátrix értékei szintén az 5 kör átlagát tükrözik, ezért látszanak kis számok, nem az összes 50 minta egyszerre kerül kiértékelésre, hanem körönként kb. 10.

A Random Forest modellnél csak a Statistical jellemzőkinyerési módszer érhető el. Ennek oka, hogy a fa alapú algoritmusok felépítéséből adódóan nagy dimenziójú jellemzővektorokon kis adathalmazon könnyen túlilleszkednek. A Phase epoch és Phase band módszerek 136, illetve 292 jellemzőt adnak vissza, ami 50 mintás adathalmazon nem szerencsés. A 33 jellemzőt tartalmazó Statistical módszer ehhez a modellhez sokkal jobban illeszkedik.

Az eredményeket az alábbi táblázat foglalja össze:

Modell	Jellemzőkinyerés	Jellemzők száma	Pontosság	Balanced accuracy
SVM	Statistical	33	90%	90%
SVM	Phase epoch	136	58%	58%
SVM	Phase band	292	56%	56%
Random Forest	Statistical	33	88%	88%
RF+MLP	Statistical	33	90%	90%
RF+MLP	Phase epoch	136	86%	86%
RF+MLP	Phase band	292	86%	86%
XGBoost	Statistical	33	88%	88%
XGBoost	Phase epoch	136	88%	88%
XGBoost	Phase band	292	88%	88%

Az eredmények alapján több fontos megfigyelés tehető. A legjobb eredményt az SVM Statistical módszerrel és az RF+MLP Statistical módszerrel érte el, mindkettő 90%-os pontossággal. Az SVM esetén azonban éles különbség mutatkozik a jellemzőkinyerési módszerek között, míg Statistical módszerrel 90%-ot ért el, Phase epoch-kal már csak 58%-ot, Phase band-del pedig 56%-ot. Ez utóbbi két eredmény közel van a véletlen találgatás szintjéhez, ami azt jelzi hogy az SVM a fázis spektrum alapú jellemzőkkel ebben az adathalmazban nem tudott értelmes osztályozást tanulni. Ennek oka valószínűleg az, hogy a fázisszögek ($-\pi$ -tól π -ig terjedő értékek) még normalizálás után is nehezebben kezelhetők az SVM számára.

Az XGBoost és az RF+MLP ezzel szemben robusztusabbnak bizonyultak. Az XGBoost mindhárom módszerrel 88%-ot ért el, az RF+MLP pedig Statistical módszerrel 90%-ot, Phase epoch és Phase band módszerekkel 86%-ot. Ez arra utal, hogy a fa alapú modellek kevésbé érzékenyek a jellemzők típusára és skálájára.

Általánosan megfigyelhető, hogy a Statistical jellemzőkinyerési módszer a legtöbb modellnél legalább olyan jó, vagy jobb eredményt ad mint a bonyolultabb fázis alapú módszerek. Ez arra utalhat, hogy az egyszerű statisztikai jellemzők, mint az átlag, szórás, teljesítménybecslés és sávarányok ebben az adathalmazban elegendő információt hordoznak az osztályozáshoz, és a fázis spektrum alapú megközelítés nem nyújt lényeges többletinformációt.

Mivel az adathalmaz tökéletesen kiegyensúlyozott, pontosan 25 beteg és 25 egészséges minta szerepel benne, a pontosság és a balanced accuracy minden esetben ugyanazt az értéket adja. Ez várható viselkedés, hiszen a balanced accuracy csak kiegyensúlyozatlan adathalmazon tér el a hagyományos pontosságtól. A táblázatban mindkét metrika szerepel a teljesség kedvéért, de az értékek azonossága megerősíti hogy az adathalmaz megfelelően van összeállítva.

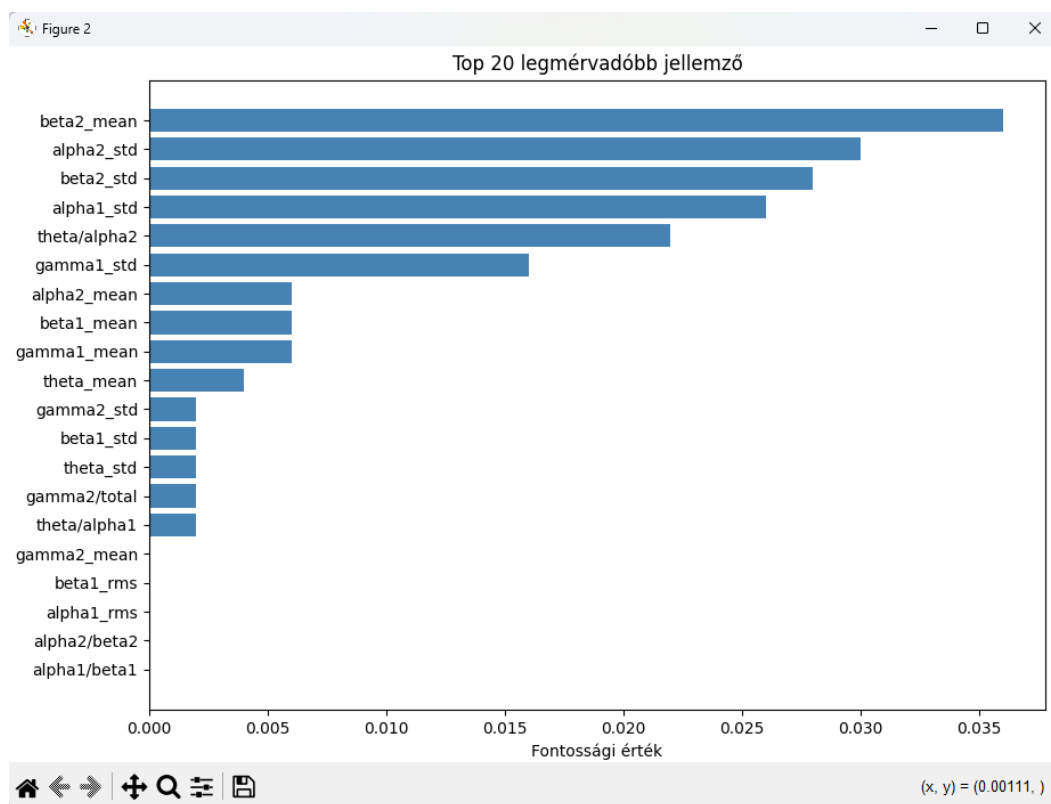
Érdemes megjegyezni, hogy a Random Forest esetén már a tervezés során tudatosan döntöttem a Statistical módszer kizárólagos használata mellett, mivel a fa alapú algoritmusok felépítéséből adódóan nagy dimenziójú jellemzővektorokon kis adathalmazon könnyen túlilleszkednek. Az XGBoost esetén ez a korlátozás nem került bevezetésre, és mindhárom módszer elérhető maradt. Az eredmények azonban utólag igazolták, hogy itt is elegendő lett volna csak a Statistical módszert engedélyezni, mivel

az XGBoost mindhárom jellemzőkinyerési módszerrel azonos, 88%-os pontosságot ért el. Ez egy hasznos tanulság a rendszer esetleges jövőbeli egyszerűsítéséhez.

3.2 Jellemzők fontossága

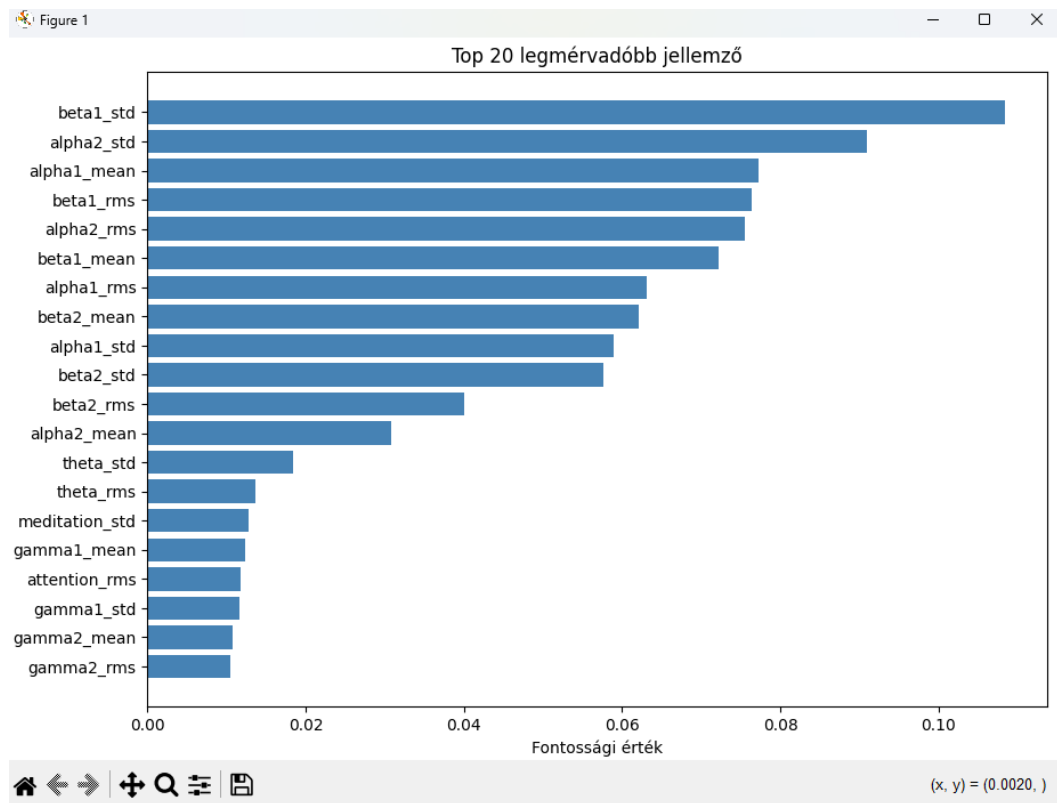
A jellemzők fontosságának vizsgálata megmutatja, hogy az egyes csatornák és statisztikai mutatók mennyire befolyásolják az osztályozási döntést. A rendszer két módszerrel határozza meg ezt: Random Forest és RF+MLP esetén a beépített `feature_importances_` tulajdonsággal, SVM esetén permutation importance módszerrel, amely 10-szer újrafuttatja a modellt, minden alkalommal egy jellemző értékeit felkeverve, és méri, mennyit esett vissza a pontosság.

Az SVM-nél a `beta2_mean` bizonyult a legfontosabb jellemzőnek, fontossági értéke meghaladja a 0.035-öt, a második helyen az `alpha2_std` áll 0.030 körüli értékkel. Ezt követi a `beta2_std` és az `alpha1_std`. Érdekes, hogy a `theta/alpha2` arány is bekerül a top 5 közé, ami arra utal, hogy a θ és α sávok egymáshoz viszonyított aránya is hordoz hasznos információt. Az `alpha1/beta1` és `alpha2/beta2` arányok, valamint az rms értékek a lista alján végeztek, ezek tehát alig befolyásolják a döntést.



3.1. ábra: Top 20 legmérvadóbb jellemző – SVM (Statistical)

Az RF+MLP-nél a beta1_std végzett az első helyen, fontossági értéke meghaladja a 0.10-et, ami jóval magasabb mint az SVM-nél látott számok. Mögötte az alpha2_std és az alpha1_mean következik. Az alfa és béta sávok jellemzői itt is dominálnak, ami összhangban van az 1.2 fejezetben leírt élettani háttérrel, ezek a sávok ugyanis aktív kognitív folyamatokhoz és relaxált éber állapothoz köthetők. Ami viszont eltér az SVM-től: itt az rms értékek is fontos szerepet kapnak, míg az SVM-nél ezek a lista végén voltak.



3.2 ábra: Top 20 legmérvadóbb jellemző – RF+MLP (Statistical)

Mindkét modellnél közös megfigyelés, hogy az alfa és béta sávok jellemzői dominálnak, a gamma sávok és a sávarányok pedig kevésbé fontosak. Ez arra utal, hogy a beteg és egészséges személyek EEG-jelei elsősorban az alfa és béta frekvenciatartományban különböznek egymástól.

3.3 Feature selection eredményei

A feature_selection.py szkripttel azt vizsgáltam meg, hogy ha visszafogom a jellemzők számát, az hogyan hat a modellek teljesítményére. A gépi tanulásban ismert jelenség, hogy felesleges vagy zajt hordozó jellemzők ronthatják az általánosító képességet, kis adathalmazon pedig különösen veszélyes, ha túl sok jellemzővel dolgozunk. Minden modellt először az összes jellemzővel futtattam le, majd a feature

importance alapján kiválasztott top 10, top 15 és top 20 legfontosabb jellemzővel, és összehasonlítottam a számokat.

A Statistical módszernél az SVM mindhárom esetben 84%-ot hozott, ami 2%-kal jobb az összes jellemzővel elért 82%-nál, tehát itt a szűkítés egyértelműen segített. A Random Forest top 10 jellemzővel 90%-ot adott a korábbi 88%-hoz képest, ami szintén javulás. Az RF+MLP top 10-zel szintén 90%-on volt, top 15 és top 20 esetén viszont visszacsúszott 88%-ra, tehát itt már nem érdemes tovább növelni a jellemzők számát. Az XGBoost top 10 és top 20 jellemzővel 90%-ot ért el, top 15-tel viszont csak 86%-ot, miközben az összes jellemzővel 88%-on állt, ami egy kissé egyenetlen képet mutat.

A Phase epoch módszernél az SVM megint 84%-ot adott mindhárom esetben, itt is 2%-kal jobb a kiindulópontnál. A Random Forest top 10, 15 és 20 jellemzővel egyaránt 88%-on zárt, ami viszont 2%-os visszaesés a 90%-hoz képest, tehát ennél a modellnél és módszernél a szűkítés nem bizonyult hasznosnak. Az RF+MLP top 10 jellemzővel 90%-ot hozott, ami 4%-os javulás a 86%-hoz képest, ez az egyik legnagyobb ugrás az egész vizsgálatban. Az XGBoost top 15 jellemzővel 90%-ot ért el szemben az összes jellemzővel elért 88%-kal, a többi konfiguráció viszont nem hozott javulást.

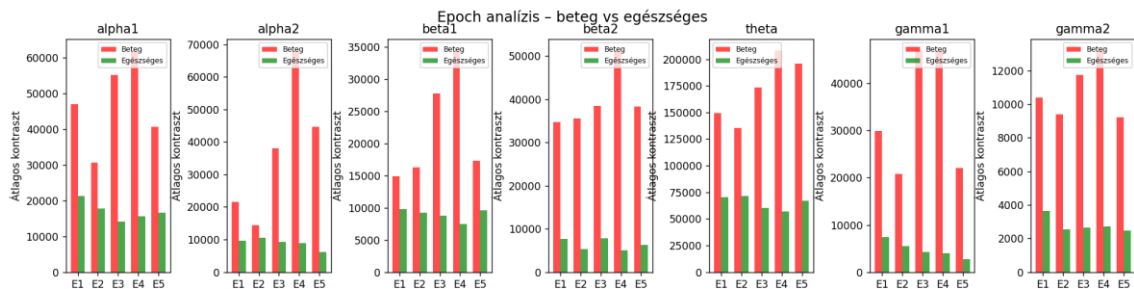
A Phase band módszernél az SVM ismét 84%-on állt mindhárom esetben, következésképpen jobbnak bizonyult az összes jellemzőnél. A Random Forest és az XGBoost top 10 és top 15 jellemzővel ugyanott voltak mint az összes jellemzővel, top 20-szal viszont mindkettő 86%-ra esett vissza, ami arra utal, hogy ennél a módszernél a top 20 már túl sok jellemzőt von be és ez ront az eredményen. Az RF+MLP top 10-zel 88%-ot hozott, ami 2%-kal jobb a 86%-nál, a többi konfiguráció viszont nem mutatott javulást.

Az összképet nézve a jellemzők visszafogása legtöbbször nem ront az eredményeken, és sok esetben kifejezetten javít rajtuk, a top 10 jellemző bizonyult a legjobb választásnak szinte minden modellnél és módszernél. Ez valószínűleg azzal magyarázható, hogy az adathalmazban viszonylag kevés jellemző hordoz ténylegesen hasznos információt, a többi inkább zajként viselkedik és megzavarja a modellt. Azt viszont érdemes szem előtt tartani, hogy ezek az eredmények egy kisebb adathalmazon születtek, és a különbségek egy része simán a keresztvalidáció véletlenszerűségéből is adódhat, tehát óvatosan kell kezelni a következtetéseket.

3.4 Epoch analízis eredményei

Az epoch_analysis.py szkripttel azt vizsgáltam meg, hogyan különböznek egymástól a beteg és egészséges csoport EEG-jelei, epochonként lebontva. Két elemzést futtattam le erre a célra: egy min-max kontraszt elemzést és egy kétmintás t-tesztet, mind a hét csatornán és mind az öt epochon.

A kontraszt elemzés lényege az amplitúdó-ingadozás mérése, és az eredmények elég egyértelműek lettek: a beteg csoport szinte minden csatornán és epochban nagyobb értékeket mutat az egészséges csoporthoz képest. A theta és béta sávokban ez különösen szembetűnő volt, egyes epochokban a különbség meghaladja a 100 000-es nagyságrendet. Ez valószínűleg azzal függ össze, hogy a beteg csoport agyi aktivitása zenehallgatás közben jóval változékonyabb.



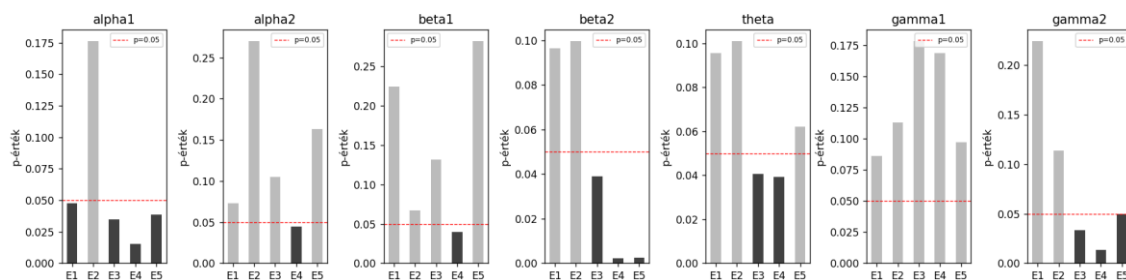
3.3 ábra: Min-max kontraszt összehasonlítása epochonként – beteg vs egészséges csoport

A t-tesztel azt néztem meg, hogy ezek a különbségek tényleg szignifikánsak-e, vagy csak véletlen ingadozásból jönnek. A határt $p < 0.05$ -nél húztam meg, és az eredmények elég érdekesek lettek.

Az alpha1 csatornán ment a legjobban a teszt, az 1, 3, 4. és 5. epochban is szignifikáns volt a különbség. A beta2 csatornán a 3, 4. és 5. epochban 0.002 és 0.004 közötti p-értékek jöttek ki, ezek a legalacsonyabb értékek az egész vizsgálatban. A gamma2 szintén a 3, 4. és 5. epochban volt szignifikáns. Az alpha2 és beta1 csak a 4. epochban, a theta a 3. és 4. epochban mutatott szignifikáns eltérést. A gamma1 egyik epochban sem adott szignifikáns különbséget.

Ami igazán feltűnő, hogy a szignifikáns különbségek nagy része a 3. és 4. epochban jön elő, tehát a felvétel közepén, nagyjából a 20-40. másodperc között. Ez arra utalhat, hogy a két csoport agyi aktivitása nem rögtön az elején tér el egymástól, hanem egy kis idő kell hozzá. Ez szerintem összhangban van azzal, hogy a zenei feldolgozás memória-

és anticipációs folyamatokat aktivál, amelyek nem azonnal, hanem fokozatosan alakulnak ki.

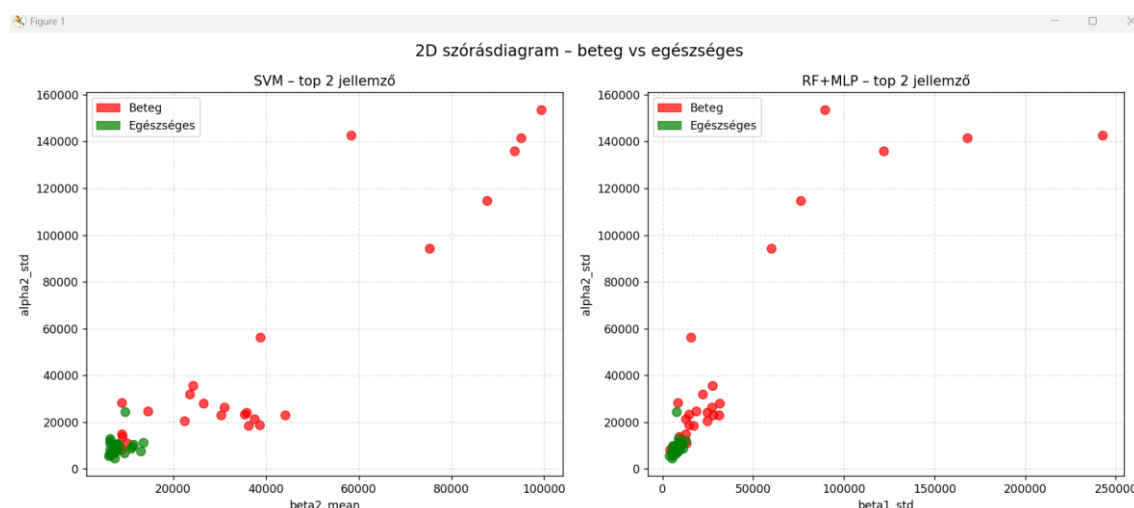


3.4. ábra: Kétmintás t-teszt p-értékei epochonként

3.5 Döntési határok és jellemzők vizualizációja

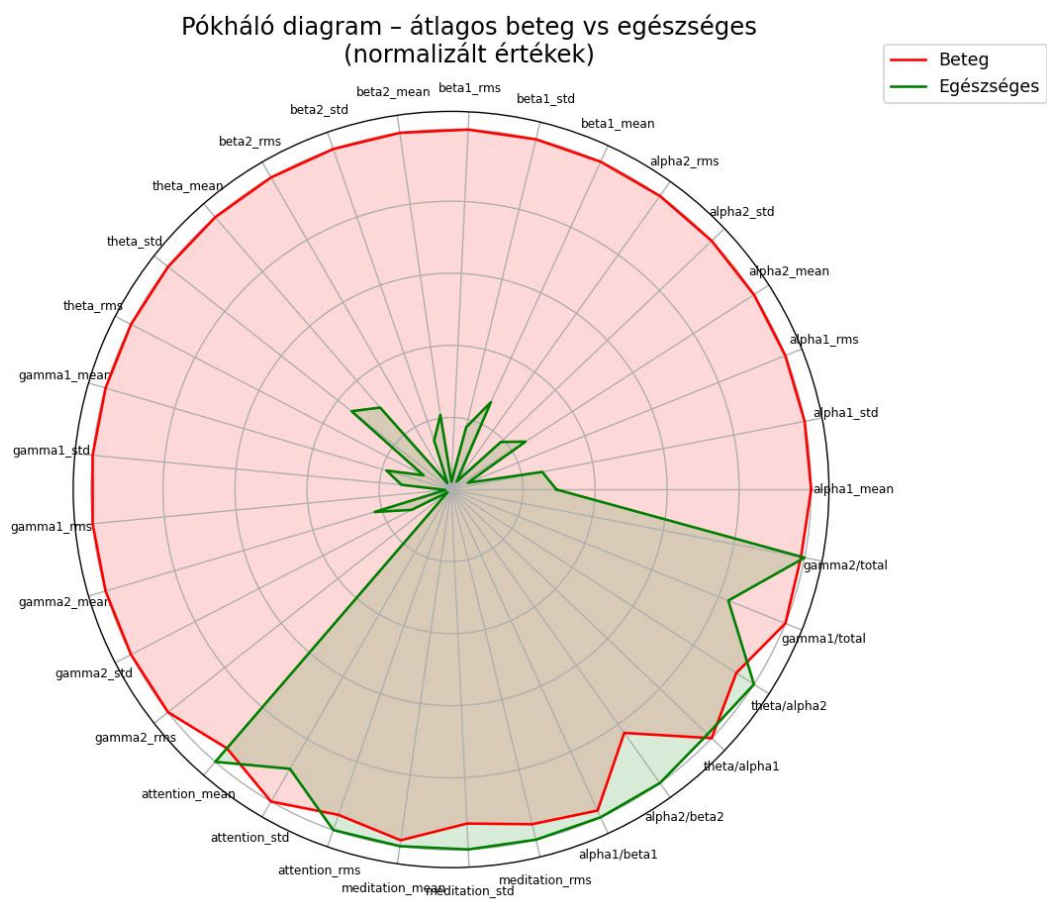
A visualization.py szkript három különböző vizualizációt készít, amelyek más-más oldalról mutatják meg, mi történik az adatokkal és hogyan dönt a modell.

Az első ábra egy koordináta-rendszerben ábrázolja a két legfontosabb jellemzőt, minden pont egy felvételt jelöl, piros a beteg, zöld az egészséges. Ami rögtön feltűnik, hogy az egészséges személyek pontjai egy szűkebb területen csomósodnak össze a bal alsó sarokban, a beteg személyek pontjai viszont szanaszét szóródnak, egészen magas értékekig elhúzódnak mindkét tengelyen. Ez nem meglepő, hiszen a kontrasztelemlésnél is ugyanez jött ki: a beteg csoport jelei változékonyabbak, és ez a jellemzőtérben is látszik. Az is kiolvasható az ábráról, hogy a beteg csoport egyedei egymástól is jobban különböznek, ami az osztályozást nyilván megnehezíti.



3.5 ábra: A beteg és egészséges csoport eloszlása a két legfontosabb jellemző mentén

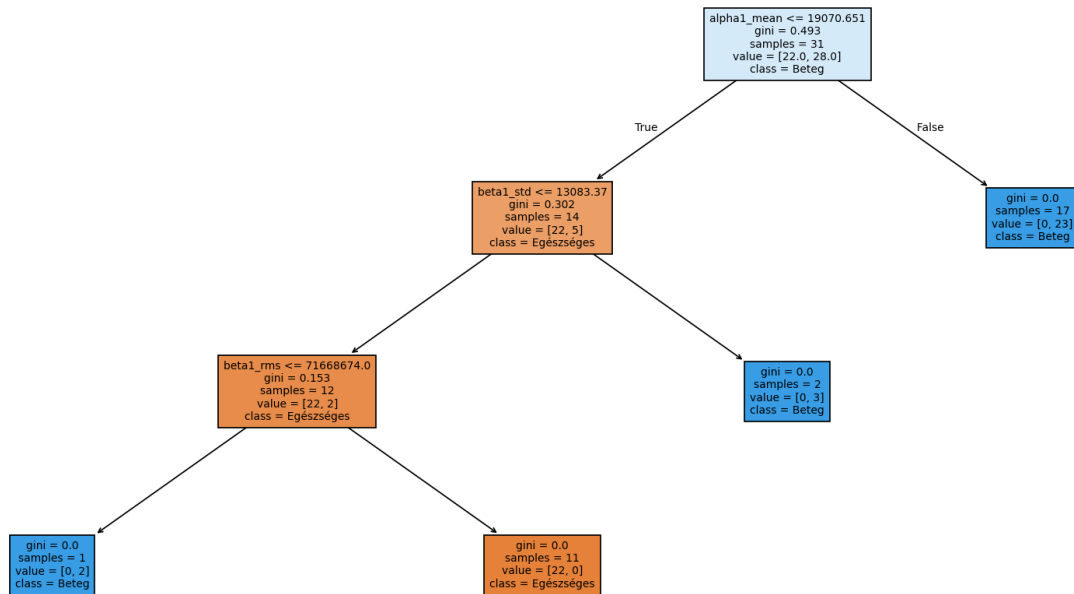
A második ábra mind a 33 jellemzőt egyszerre mutatja be egy pókháló diagramon, a piros vonal a beteg csoport átlagát, a zöld az egészséges csoport átlagát jelöli normalizált értékeken. Az eredmény elég egyértelmű: a zöld vonal szinte minden tengelyen jóval beljebb van mint a piros – tehát az egészséges csoport jellemzőértékei átlagosan alacsonyabbak. Kivételt képeznek a sávarányok – mint az α_1/β_1 , α_2/β_2 , θ/α_1 , ahol a zöld vonal kicsit kijebb kerül, ami azt jelenti hogy ezeknél a relatív arányoknál az egészséges csoport mutat magasabb értékeket. Ez megerősíti amit a korábbi elemzésekből is láttam: a beteg csoport abszolút értékei nagyobbak, de a sávok egymáshoz viszonyított arányaiban más a kép.



3.6 ábra: A beteg és egészséges csoport jellemzőinek összehasonlítása pókháló diagramon

A harmadik ábra a Random Forest modell egyik döntési fáját mutatja be 3 szint mélységig. A gyökerécsomópontban az α_1_mean jellemző szerepel 19070.651-es küszöbértékkel – tehát a modell először azt vizsgálja, hogy az α_1 sáv átlagos értéke kisebb-e ennél a küszöbnél. Ha igen (True ág), a modell tovább vizsgálja a β_1_std

értékét, ha nem (False ág), akkor közvetlenül betegnek osztályozza a felvételt és ez az ág 17 mintából 23-at helyesen sorolt be betegnek. A True ágon tovább haladva a beta1_rms értéke alapján dönt, és az egészséges osztályba sorolja a kis értékű mintákat. A fa kék csomópontjai a beteg, a narancssárga csomópontjai az egészséges osztályt jelölik. Ez persze csak egyetlen fa a sok közül – a végső döntést az összes fa szavazása alapján hozza meg a modell, szóval ez az ábra inkább csak egy betekintés a modell gondolkodásába.



3.7 ábra: A Random Forest modell egyik döntési fája

3.6 PCA vizualizáció

A PCA lényege, hogy a 33 jellemzőt 2 dimenzióra tömöríti le, és az így kapott ábrán meg lehet nézni, hogy a két csoport hogyan helyezkedik el egymáshoz képest. Log skálát kellett használni, mert a jellemzőértékek nagyságrendekben különböznek egymástól, lineáris skálán szinte semmi sem látszana.

Az ábra alapján az egészséges személyek pontjai nagyjából egy helyen maradnak, a beteg személyek pontjai viszont sokkal szétszórtabbak. A diagram jobb oldali és felső részein vannak olyan piros pontok, amelyek teljesen kilógnak a zöld csoportból. Ez nem meglepő, a kontraszt elemzésnél és a 2D szórásdiagramnál is ugyanez jött ki.

Ami viszont szintén jól látszik, hogy a két csoport nem válik szét élesen. A középső részen piros és zöld pontok keverednek egymással, és ez szerintem jól megmagyarázza, miért nem éri el egyik modell sem a 100%-ot. Van egy olyan tartomány, ahol a beteg és

egészséges személyek jellemzőértékei nagyon hasonlítanak egymásra, és ezt a modellek sem tudják maradéktalanul kezelni.

4. Felhasználói és telepítési útmutató

4.1 Rendszerkövetelmények

A rendszer futtatásához Python 3.8 vagy újabb verzió szükséges. A fejlesztés és tesztelés Python 3.11-es verzióján történt, ezért ezt ajánlott használni. A következő külső könyvtárak telepítése szükséges:

- numpy – numerikus számítások
- pandas – CSV fájlok beolvasása és kezelése
- scikit-learn – gépi tanulási modellek és kiértékelési metrikák
- xgboost – XGBoost algoritmus
- matplotlib – grafikonok és vizualizációk
- scipy – t-teszt és statisztikai számítások
- joblib – modellek mentése és betöltése
- tkinter – grafikus felhasználói felület

A rendszer Windows operációs rendszeren lett fejlesztve és tesztelve, de Python kompatibilitásának köszönhetően Linux és macOS rendszereken is futtatható.

4.2 Telepítési útmutató

Ha még nincs Python telepítve, a legújabb verzió letölthető a hivatalos oldalról: <https://www.python.org/downloads/>. Telepítés során fontos bejelölni az „Add Python to PATH” opciót, különben a parancssor nem fogja megtalálni a Python futtatókörnyezetet.

A projekt fájljait a 2.1 fejezetben bemutatott mappastruktúra szerint kell elhelyezni, az összes Python fájl egy közös mappába kerül, amelyen belül létre kell hozni a data mappát a beteg, egészséges és test almappákkal. A beteg és egészséges mappákba kell elhelyezni a tanítóadatokat, a test mappába pedig a manuális tesztelésre szánt felvételeket.

A szükséges könyvtárakat a következő paranccsal lehet telepíteni a parancssorból vagy egy preferált IDE használatával futtatni például PyCharm beépített terminálján:

```
pip install numpy pandas scikit-learn xgboost matplotlib  
scipy joblib
```

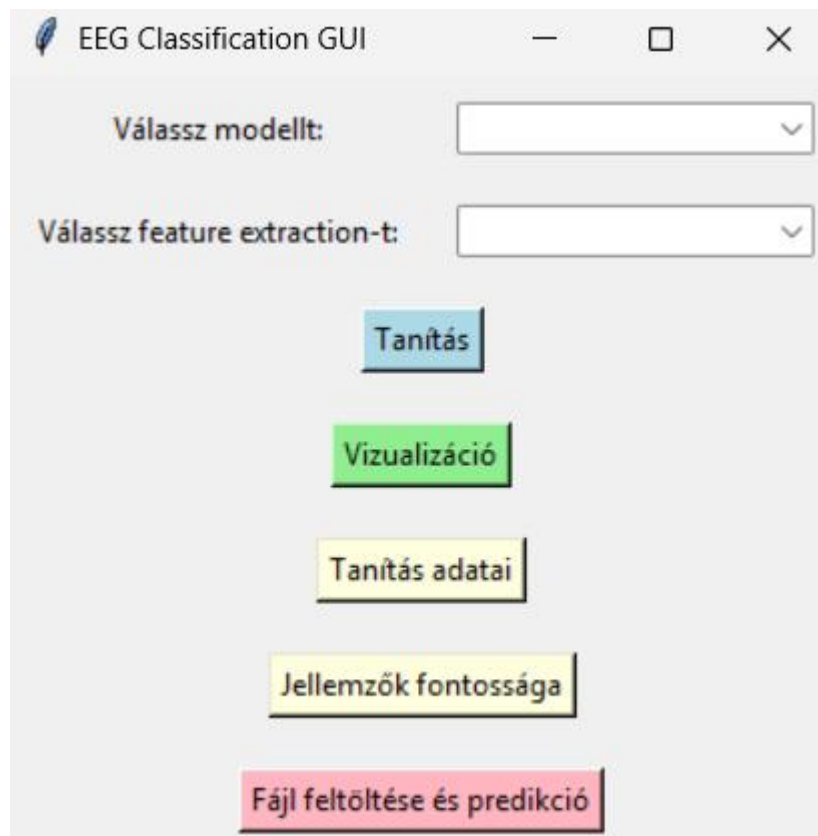
A grafikus felület a gui.py fájl futtatásával indítható el:

```
python gui.py
```

PyCharmban elegendő megnyitni a gui.py fájlt és a zöld lejátszás gombra kattintani. Az ablak megnyitása után a rendszer használatra kész, a modellek betanítása előtt nincs szükség semmilyen előkészületre.

4.3 Grafikus felület használata

A grafikus felület elindítása után az alábbi ablak jelenik meg:



4.1 ábra: A grafikus felhasználói felület főablaka

A felület két legördülő menüből és öt gombból áll. Az első legördülő menüben a modellt lehet kiválasztani, SVM, Random Forest, Random Forest MLP vagy XGBoost közül, a másodikban a jellemzőkinyerési módszert, Statistical, Phase epoch vagy Phase band közül. Fontos, hogy Random Forest modell esetén a jellemzőkinyerési módszer automatikusan Statistical-ra áll és nem változtatható meg, mivel ez a modell csak ezzel működik megfelelően.

A Tanítás gomb elindítja a kiválasztott modell betanítását. A tanítás a háttérben fut, és befejezésekor egy felugró ablak jelez. Ezt a lépést kell először elvégezni, a többi gomb csak sikeres tanítás után használható.

A Vizualizáció gomb egy PCA alapú szórásdiagramot jelenít meg, amely megmutatja, hogyan helyezkednek el a beteg és egészséges minták a jellemzőtérben. A diagram log skálán jelenik meg a jellemzőértékek nagy eltérése miatt.

A Tanítás adatai gomb egy külön ablakban mutatja meg a legutóbbi tanítás eredményeit, a minták számát, a pontosságot, a balanced accuracy-t és a konfúziós mátrixot.

A Jellemzők fontossága gomb egy vízszintes oszlopdiagramon mutatja a top 20 legfontosabb jellemzőt fontossági értékük szerint csökkenő sorrendben, tehát megmutatja, hogy a modell számára melyik EEG csatorna és statisztika bizonyult a legmértékadóbbnak.

A Fájl feltöltése és predikció gomb lehetővé teszi, hogy egy új EEG felvételt töltsünk be. A fájlnek .csv kiterjesztésűnek kell lennie és ugyanolyan szerkezetűnek mint a tanítóadatok. A betöltés után a rendszer azonnal megmondja, hogy a felvétel alapján a személy betegnek vagy egészségesnek osztályozható.

4.4 Az elemző szkriptek használata

A rendszer három önállóan futtatható elemző szkriptet tartalmaz, amelyek a GUI-tól teljesen függetlenül működnek. Ezeket közvetlenül PyCharmból lehet elindítani a zöld lejátszás gombra kattintva, vagy parancssorból a python fajlnev.py paranccsal.

Az epoch_analysis.py futtatása előtt nincs szükség előzetes tanításra, a szkript közvetlenül a data/beteg és data/egeszes mappákból olvassa be az adatokat. Futtatás után a konzolra kiírja az összes csatorna és epoch kontraszt és t-teszt eredményeit, majd megjelenít egy grafikont, amelynek felső sora a min-max kontrasztot, alsó sora a t-teszt p-értékeit mutatja.

A feature_selection.py szintén közvetlenül az adatmappákból dolgozik, de mivel minden modellt többször is betanít, a futási ideje jelentősen hosszabb lehet, ezért erősebb gépen ajánlott futtatni. Az eredmények a konzolra kerülnek kiírásra, és megmutatják, hogy top 10, 15 vagy 20 jellemzővel javítható-e a modellek teljesítménye.

A visualization.py futtatásához előbb a GUI-ban be kell tanítani egy Random Forest modellt, mivel a döntési fa vizualizációhoz szükség van az elmentett rf_model.pkl fájlra. A szkript egymás után jeleníti meg a három vizualizációt, a 2D szórásdiagramot, a pókháló diagramot és a döntési fát, és minden ábra bezárása után következik a következő.

5. Összefoglalás

A szakdolgozat célja egy olyan EEG-alapú osztályozórendszer fejlesztése volt, amely képes szétválasztani halmozottan fogyatékos és egészséges személyek felvételeit agyi elektromos aktivitásuk alapján. Az adatok Dr. Tiszai Luca kutatásához kötődnek, a felvételek zenehallgatás közben készültek, és összesen 51 tanítóadat állt rendelkezésre.

Három jellemzőkinyerési módszert valósítottam meg. Az `extract_statistical` egyszerű statisztikai jellemzőkre épül, az `extract_phase_epoch` epochonkénti fázis spektrum elemzést végez, az `extract_phase_band` pedig frekvenciasávonként szűrt fázis jellemzőket nyer ki. A kinyert jellemzők alapján négy modellt tanítottam be és hasonlítottam össze, SVM-et, Random Forest-et, Random Forest MLP-t és XGBoost-ot, 5-szörös rétegzett keresztvalidációval.

A legjobb eredményt az RF+MLP és az SVM érte el Statistical módszerrel, mindkettő 90%-ot hozott. Az XGBoost és a Random Forest 88%-on zárt. Ami igazán feltűnő, hogy az SVM fázis alapú jellemzőkkel mindössze 56-58%-ot ért el, ami közel van a véletlen találgatás szintjéhez. Ez arra utal, hogy az SVM nehezebben boldogul a fázisszögekkel mint a fa alapú modellek. Az is kijött, hogy a Statistical módszer a legtöbb modellnél legalább olyan jó eredményt adott mint a bonyolultabb fázis alapú módszerek, szóval az egyszerű statisztikai jellemzők elegendő információt hordoznak az osztályozáshoz.

A feature importance elemzésből az jött ki, hogy az SVM esetén a `beta2_mean` és az `alpha2_std` a legfontosabb jellemzők, az RF+MLP-nél pedig a `beta1_std` és az `alpha2_std`. A feature selection vizsgálat azt mutatta, hogy top 10 jellemzővel több modellnél is javult a pontosság, ami megerősíti, hogy a 33 jellemző közül viszonylag kevés hordoz ténylegesen hasznos információt.

Az epoch analízis szerint a beteg csoport szinte minden csatornán és epochban nagyobb min-max kontrasztot mutat az egészséges csoporthoz képest, és a t-teszt alapján ez több csatornán szignifikáns is, különösen az `alpha1` és `beta2` sávokban. A szignifikáns különbségek főként a 3. és 4. epochban jönnek elő, ami arra utalhat, hogy a két csoport agyi aktivitása nem azonnal, hanem egy kis idő elteltével kezd el szignifikánsan eltérni.

A rendszernek persze vannak korlátai is. Az 51 tanítóadat viszonylag kevés, ami megnehezíti az általánosítást és befolyásolhatja az eredmények megbízhatóságát. Az

SVM normalizálás nélkül nem konvergált rendesen, tehát az előfeldolgozás komoly szerepet játszik. A fázis alapú módszerek nem hoztak érdemi javulást, ami részben azzal magyarázható, hogy az adatokban szereplő frekvenciasáv értékek már eleve előfeldolgozott adatok, nem nyers EEG idősorok.

Jövőbeli fejlesztési lehetőségként több adat bevonása sokat segíthetne. Érdeemes lenne nyers EEG idősorokat is vizsgálni, ahol a fázis spektrum elemzése valódi hozzáadott értéket jelenthet. Mélyebb neurális hálók alkalmazása és a feature_selection eredményeinek beépítése a fő rendszerbe szintén javíthatna a teljesítményen.

6. Nyilatkozat AI használatáról

Eszköz neve	Felhasználás célja	Felhasználás módja
Claude (Sonnet 4.6)	Képgenerálás	Képek generálása ábrákhoz
Claude (Sonnet 4.6)	Szöveg ellenőrzése	Nyelvtani és formai helyesség ellenőrzése

Dátum 2026.05.22

Szabó-Szűcs Benárd

Aláírás

Irodalomjegyzék

- [1] Miltiadous, A., Tzamourta, K. D., Afrantou, T., Ioannidis, P., Grigoriadis, N., Tsalikakis, D. G., Angelidis, P., Tsipouras, M. G., Glavas, E., Giannakeas, N., & Tzallas, A. T. (2023). *Parkinson's Disease EEG Dataset*. OpenNeuro. <https://openneuro.org/datasets/ds004504/versions/1.0.8>
- [2] Tiszai, L. (2024). A zeneterápia mint közösségi zeneterápia a gyógypedagógiában. In Fekete Zsófia (szerk.), *Széles spektrumú zeneterápia* (pp. 159–171). ISBN: 9789632269290.
- [3] Zabihi, M. et al. (2019). *Application of Machine Learning in Epileptic Seizure Detection*. PubMed Central. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9689720/>
- [4] Seal, A. et al. (2024). *A machine learning based depression screening framework using temporal domain features of the electroencephalography signals*. PLOS One. <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0299127>
- [5] Shumbayawonda, E. et al. (2023). *Computational methods of EEG signals analysis for Alzheimer's disease classification*. Scientific Reports. <https://www.nature.com/articles/s41598-023-32664-8>
- [6] Newson, J. J., & Thiagarajan, T. C. (2019). EEG Frequency Bands in Psychiatric Disorders: A Review of Resting State Studies. *Frontiers in Human Neuroscience*, 12, 521. <https://doi.org/10.3389/fnhum.2018.00521>
- [7] Anthropic. (2026). *Az EEG öt fő frekvenciasávja* [AI-generált ábra]. Claude Sonnet 4.6. <https://claude.ai>. Prompt: "Készíts egy ábrát az EEG öt fő frekvenciasávjáról (delta, theta, alfa, beta, gamma), minden sávnál tüntesd fel a frekvenciatartományt és az élettani jelentést, valamint egy szemléltető hullámformát. Magyar nyelven, tiszta, letisztult stílusban."
- [8] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
- [9] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- [10] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- [11] Anthropic. (2026). *Az 5-szörös rétegzett keresztvalidáció működése* [AI-generált ábra]. Claude Sonnet 4.6. <https://claude.ai>. Prompt: "Rajzolj egy ábrát az 5-szörös rétegzett keresztvalidáció működéséről, ahol minden körben más fold kerül tesztelésre és a többi tanításra szolgál. Fekete-fehér, tiszta stílusban, magyar feliratokkal."

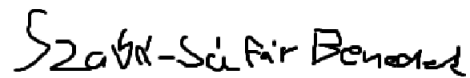
Nyilatkozat

Alulírott Szabó-Sáfár Benedek Programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Képfeldolgozás és Számítógépes Grafika Tanszékén készítettem, Programtervező informatikus BSC diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Dátum 2026.05.22



Aláírás